# DefendIt

## Thesis Consultation and Milestone Tracker

Project Documentation and System Guide

FLUTTER ELECTIVE 3 FINAL PROJECT

Prepared by: _____

Course and section: _____

Instructor: _____

Date submitted: _____

Flutter • Dart • Firebase Authentication • Cloud Firestore • SQLite

This guide describes DefendIt's purpose, users, requirements, interface, architecture, database, authentication, synchronization, installation, and limitations. It follows the ten core sections of the project documentation.

# 1 Project Overview

## 1.1 Introduction

DefendIt is a Flutter and Firebase mobile application that supports thesis advising between student groups and their advisers. A group submits title ideas, books consultation slots, and tracks its work through six thesis milestones. The adviser accepts groups, reviews titles, records consultation notes and action items, and approves milestones.

## 1.2 Problem and Objectives

Thesis consultations are often coordinated through chat messages and paper notes. Agreements can be forgotten, feedback becomes scattered, and progress is difficult to follow. DefendIt aims to:

- Let a leader register a group with its section, topic, and two to five members, and let advisers accept up to four groups.
- Let students submit title ideas containing a title, description, and purpose for adviser review.
- Let advisers publish consultation slots that groups can book without double booking.
- Record consultation notes and assignable action items.
- Let groups plan milestone tasks with assignees, due dates, and progress indicators.
- Keep core records available offline in a local database that synchronizes with the cloud.

## 1.3 Compliance with Project Requirements

| Requirement | How DefendIt addresses it |
|---|---|
| 2a Core purpose | Thesis groups and advisers; lost agreements, scattered feedback, and unclear progress. |
| 2b Unique UI and UX | Custom components and interaction states (Section 4). |
| 2c Authentication | Google sign-in, roles, sessions, and registration checks (Section 7). |
| 2d Database | Live Firestore listeners and local SQLite storage (Sections 6 and 8). |
| 2e Topic exclusivity | Thesis-advising workflow; topic approval is handled through the instructor's approval process. |
| Mobile application | Android release APK with CRUD (Section 6.4) and authentication. |
| Documentation | This guide and a separate itemized Test Cases Document (Section 10.3). |

# 2 Target User Personas

These sample personas illustrate the intended users; they are not presented as interview findings.

| Persona | Needs | How DefendIt helps |
|---|---|---|
| Group leader Ana | Coordinate a group of up to five, gain title approval, and prepare for meetings. | Title feedback, bookable slots, action items, and milestone tasks. |
| Group member Ben | Join the group and see assigned work and progress. | Student-number registration, teammate code, and task assignees. |
| Adviser Engr Reyes | Review up to four groups and retain a record of agreements. | Request previews, review tools, notes, and group progress. |

# 3 Requirements

## 3.1 Functional Requirements

| Feature | Required behavior |
|---|---|
| Sign-in and roles | Google sign-in and a one-time role choice. Advisers claim an access code; students register a unique student number. |
| Groups | One group per student. The leader lists two to five members. Teammates join using a six-character code during sign-up or later. |
| Adviser requests | Preview an open group's name, section, topic, members, and up to three title ideas; accept up to four groups. |
| Title ideas | Members create proposals and revise permitted ideas. Advisers choose Approved, Needs polish, or Rejected; feedback explains required changes. |
| Scheduling | Advisers publish future non-overlapping slots. Students book slots; either side can cancel eligible bookings. Transactions prevent double booking. |
| Consultations | Advisers record notes and action items; group members mark action items complete. |
| Milestones | Six milestones per group. Members assign and complete tasks, then submit eligible work. Advisers approve, request revision, and set due dates. |
| Dashboard | Today shows the next consultation, counts, and a Needs attention list. |
| Offline records | Notes, action items, title ideas, milestones, and tasks are stored locally and synchronized. A chip displays sync status. |

## 3.2 Non Functional Requirements

Usability requires a consistent theme and distinct loading, empty, error, success, and validation states. Forms should make incomplete or invalid input visible before submission.

Security requires authenticated access and Firestore Security Rules that restrict records according to profile ownership, group membership, and assigned adviser. Client controls and server rules serve different purposes: hiding an action does not replace backend authorization.

Integrity relies on transactions for operations that compete for shared records, including booking and adviser acceptance. The limits and enforcement boundaries are discussed in Section 10.1.

Performance uses SQLite for local records, so already downloaded work can be displayed and edited without waiting for a cloud round trip. Coordination features still access Firestore directly.

Compatibility targets an Android release APK. The source documentation identifies Android 6.0 or later as the intended baseline; the final supported minimum must also satisfy the Flutter and Firebase dependencies used by the release build. Flutter's cross-platform framework does not establish that every platform has been configured or tested.

# 4 UI Design and Wireframes

## 4.1 Visual Identity

The source design uses deep indigo for headers and primary actions, amber for highlights, paper white for backgrounds, green for success, and red for errors. The wireframes retain these visual roles. Their sample names and example study titles illustrate screen content rather than additional DefendIt features.

This publication uses Century Gothic throughout. The source describes Source Serif 4 headings and Inter body text inside the application; the diagrams below use Century Gothic to meet the publication typography requirement.

Rounded cards, status chips, bottom sheets, progress bars, and a sync chip form the main component set. Loading, empty, error, and validation states are part of each relevant flow.

## 4.2 Navigation Structure

After Google sign-in and one-time role setup, the user enters a shell with four tabs.

| Tab | Student | Adviser |
|---|---|---|
| Today | Next consultation, counts, and Needs attention. | The dashboard from the adviser's perspective. |
| Schedule | Available slots and My bookings. | Availability list and Add slot. |
| Groups | My group, create or join, and title ideas. | My groups, Requests, previews, and title review. |
| Progress | Milestones and task screens. | Group progress, approvals, and revision requests. |

## 4.3 Wireframes

The first two wireframes show entry into the application. Sign-in establishes identity; role setup collects the details needed to use the student or adviser workflow.

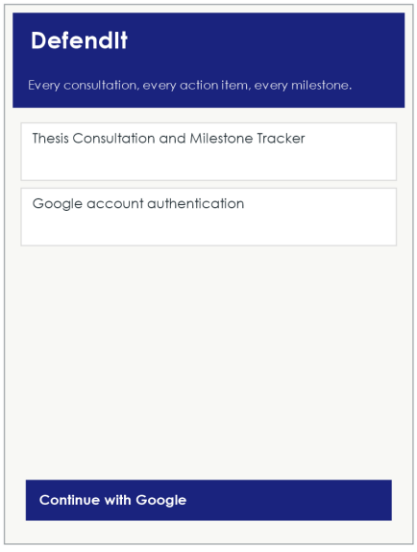



*Figure 4.1 Login*

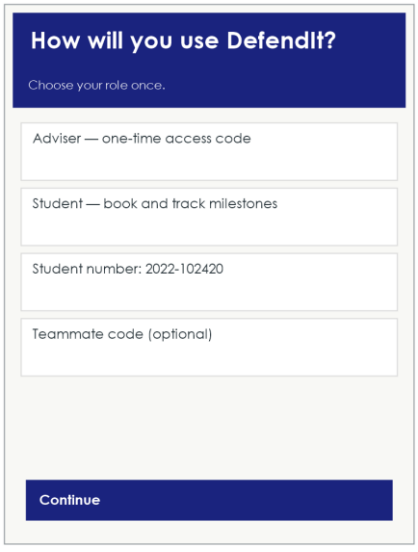



*Figure 4.2 Student registration*

## 4.3 Wireframes continued

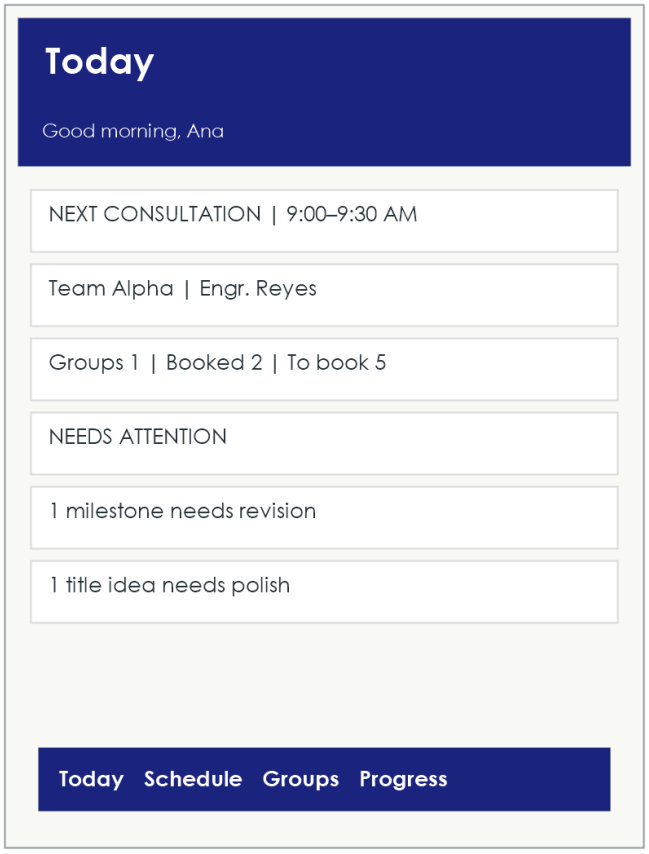



*Figure 4.3 Today dashboard*

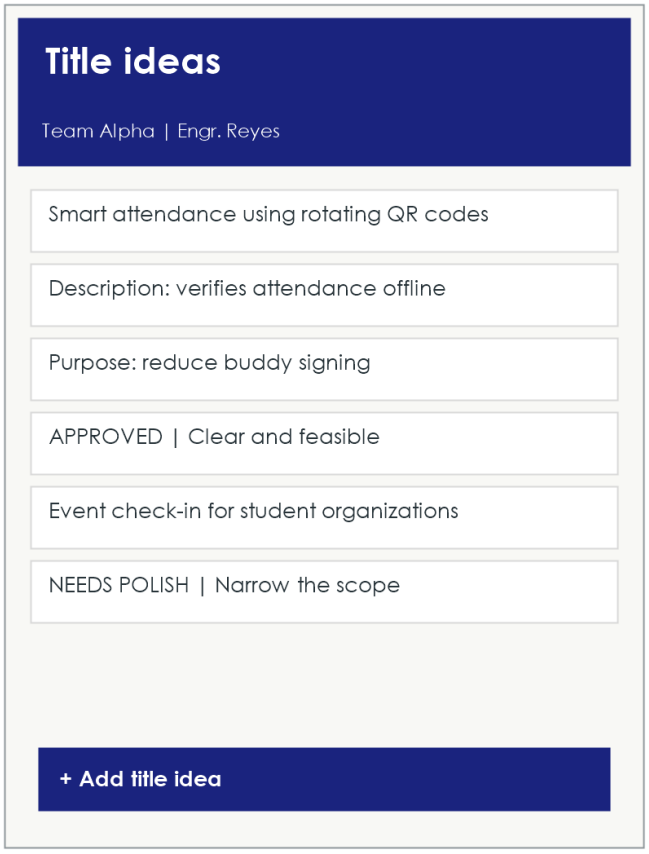



*Figure 4.4 Title ideas*

The Today dashboard prioritizes the next consultation and work needing attention. Counts summarize the user's groups and consultation availability. The sync chip communicates whether local work is offline, uploading, or synchronized; it is not an adviser-approval indicator.

The Title ideas screen separates the proposed title, description, and purpose from the review result. Approved, Needs polish, and Rejected are adviser decisions. Feedback tells the group what needs to change before another review.

The example title ideas in Figure 4.4 are sample thesis topics entered by a group. They do not expand DefendIt's scope into attendance tracking. DefendIt's function is to manage their submission and review.

## 4.3 Wireframes continued

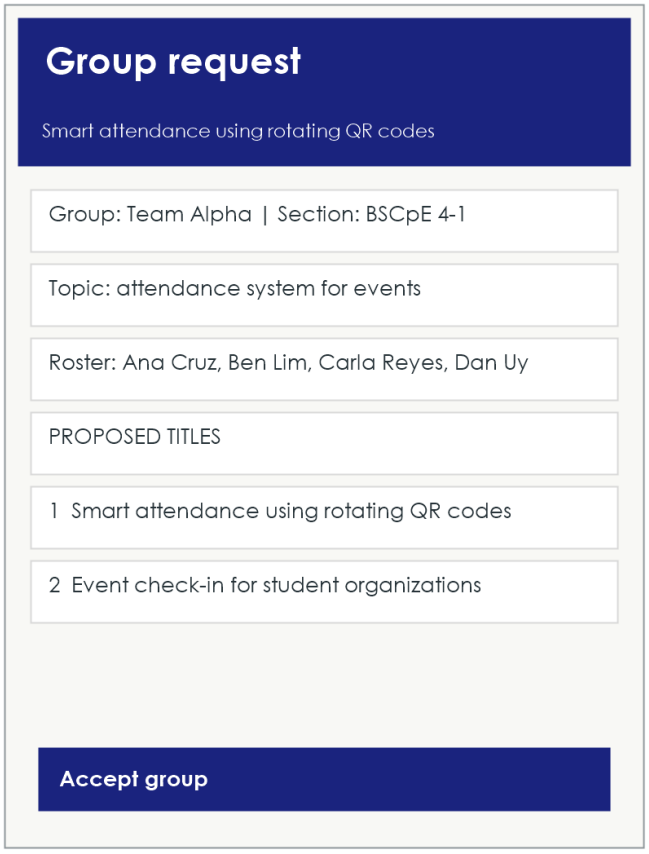



*Figure 4.5 Adviser preview*

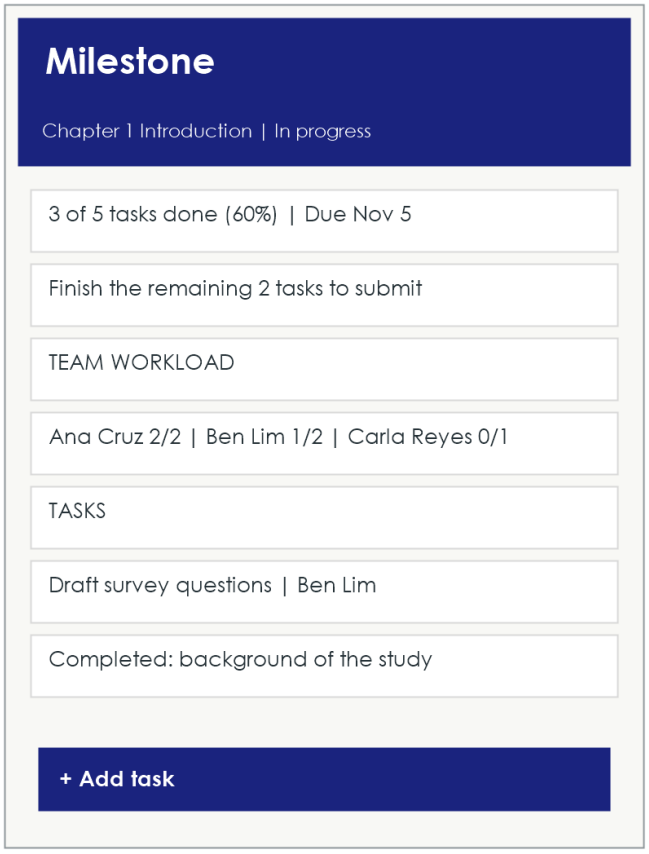



*Figure 4.6 Milestone tasks*

The adviser preview brings together a group's section, thesis topic, roster, and proposed titles before acceptance. It supports the adviser-request workflow described in Section 3.1, including the limit of four accepted groups.

The milestone task screen shows the milestone status, task completion, due date, assignees, and team workload. Students submit a milestone only after its tasks meet the submission conditions. Adviser approval remains separate from completing tasks.

Status, completion counts, and feedback should be read together. A task progress bar describes work completed; a milestone approval records the adviser's decision. Revisions return attention to the work that still needs to be addressed.

# 5 System Architecture

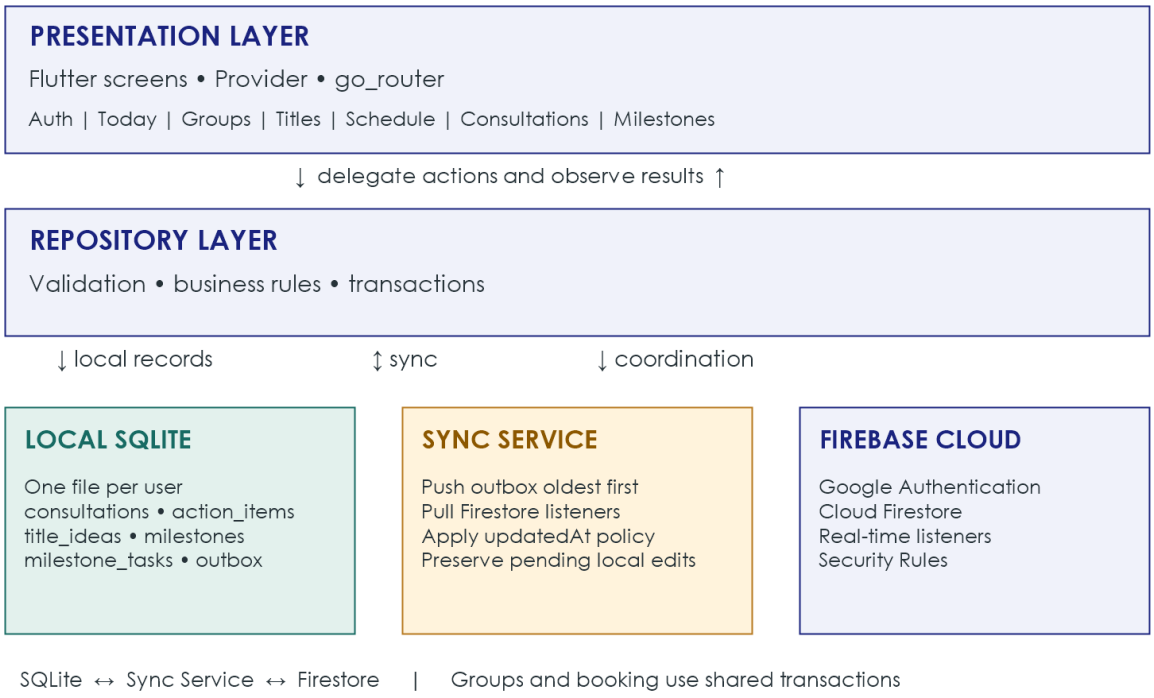



*Figure 5.1 System architecture*

The presentation layer contains Flutter screens grouped by feature. Provider supplies state, and go_router handles navigation. Screens delegate persistence to repositories rather than reading or writing databases directly.

The repository layer holds feature validation, business rules, and transactions. Authentication, groups, slots, titles, consultations, milestones, and attention summaries each have a repository responsibility.

Offline-first records—notes, action items, titles, milestones, and tasks—are read from SQLite. Local writes save the data change and its outbox entry together. The sync service sends queued operations to Firestore when online and applies incoming listener updates to the local database.

Coordination features, including groups, slots, booking, and authentication, use Firebase directly. Shared-record transactions prevent competing booking or acceptance operations from being treated as independent local edits. Section 8 explains the synchronization boundaries.

The main packages are firebase_auth, google_sign_in, cloud_firestore, sqflite, provider, go_router, and connectivity_plus. SQLite provides per-user local storage, while Cloud Firestore distributes shared records through real-time listeners.

# 6 Database Schema

## 6.1 Entity Relationship Diagram




*Figure 6.1 Entities fields and relationships*

Identifiers connect the entities shown above. A group contains title, milestone, and task subcollections. A consultation reuses its booked slot's identifier, and action items reference their consultation and group. These are stored document references by identifier, rather than relational foreign-key constraints enforced by Firestore.

The diagram shows the source's entity fields. Profile and registration timestamps use Firestore server timestamps; operational date fields such as startsAt, dueAt, and updatedAt use integer milliseconds since the Unix epoch. Lists and maps retain their Firestore types; local serialization is described in Section 6.3.

## 6.2 Cloud Firestore Collections

| Collection or document path | Purpose |
|---|---|
| users/{uid} | Profile and role, student number or adviser code, and adviser group count. |
| studentNumbers/{number} | Reserves a normalized student number for one account. |
| adviserCodes/{code} | Pre-created adviser access codes and their claim details. |
| groups/{leaderUid} | Name, section, topic, roster, account membership, join code, and adviser. |
| groups/{id}/titles/{titleId} | Proposals, review status, feedback, and review metadata. |
| groups/{id}/milestones/{milestoneId} | Six milestones, their status, dates, and feedback. |
| groups/{id}/tasks/{taskId} | Tasks, assignee, due date, completion, and completion author. |
| slots/{slotId} | Adviser availability and booking information. |
| consultations/{slotId} | Notes and session details for a booked slot. |
| actionItems/{itemId} | Assigned consultation actions, due date, and completion. |

## 6.3 Local SQLite Database

Each signed-in user has a file named thesistrack_{uid}.db. Tables are consultations, action_items, title_ideas, milestones, milestone_tasks, and outbox. The outbox holds changes waiting to be sent to Firestore. Local identifiers are TEXT primary keys; done is stored as 0 or 1, and the consultation roster is JSON text locally rather than a Firestore list.

## 6.4 CRUD Operations

| Entity | Create | Read | Update | Delete |
|---|---|---|---|---|
| Group | Student once | Members; adviser or request preview | Join; adviser accepts | Not provided |
| Title idea | Member | Members; adviser or request preview | Pending or Needs polish revision; adviser review | Member if not approved |
| Milestone | With group | Members; adviser | Work status; review; due date | Not provided |
| Task | Member | Members; adviser | Member while unlocked | Member while unlocked |
| Slot | Adviser | Adviser; relevant students | Book; cancel | Owner if open |
| Consultation | Adviser | Adviser; members | Adviser notes | Not provided |
| Action item | Adviser | Adviser; members | Completion by members or adviser | Adviser |

# 7 Firebase Configuration and Authentication

Google sign-in uses Firebase Authentication. It avoids the SMS delivery and quota concerns of phone authentication and supports testing with several Google accounts.

## 7.1 Setup Steps

1. Create a Firebase project and register an Android app with the application's package name. Enable Google in Authentication.

2. Register the signing key's SHA-1 fingerprint. Run .\gradlew signingReport from the android folder in PowerShell. The source release configuration uses the debug key; register the actual release key if that changes.

3. Run flutterfire configure, place google-services.json in android/app, and set the Web client ID in lib/core/constants/strings.dart.

4. Create Cloud Firestore and publish the project's Security Rules. Pre-create the adviserCodes documents with the document ID equal to the issued code. The source describes an initial set of 15 codes; the number is a provisioning choice.

## 7.2 Roles Identity and Sessions

One-time roles are adviser or student. Advisers enter an issued code (CPE-XXXX-XXXX); the profile and claim are saved in one transaction. Students register a unique student number and may enter a teammate code to join a group during sign-up.

Student numbers use a four-digit year beginning with 20 and six further digits, displayed as 202X-XXXXXX. Sessions persist across restarts; sign-out clears Google and Firebase sessions.

## 7.3 Security Rules Summary

Rules restrict profile access, protect identity fields, and authorize group records by membership or assigned adviser. Students cannot approve milestones or review titles. Tasks lock while a milestone is under review or approved. These are required backend policies; their deployed enforcement must be tested, including the limits noted in Section 10.1.

# 8 Local Database and Synchronization

SQLite stores local records, Cloud Firestore stores shared records, and the sync service connects them. Push saves a local change and its outbox operation in one SQLite transaction, then replays operations oldest first. Updates send changed fields; new records need their full payload. Failed operations remain queued for a later retry.

Pull uses Firestore listeners to update SQLite and refresh screens. Incoming changes do not replace a record with pending local edits; otherwise updatedAt determines whether an older incoming value is retained or applied. Device-clock ordering is a limitation, not guaranteed conflict resolution.

Loaded notes, action items, titles, milestones, and tasks work offline. Sign-in, group coordination, slot changes, booking, and initial consultation opening need internet. Today combines local and remote data, so its entire view is not guaranteed offline. The chip shows Offline, Syncing, or Synced; tapping it requests sync when connected.

# 9 Setup and Installation Instructions

Prerequisites are Flutter SDK, Android Studio and Android SDK, Node.js and Firebase CLI, FlutterFire CLI, and an Android phone or emulator with Google Play. The device must meet the release build's minimum Android version and dependency requirements (Section 3.2).

## 9.1 Run in Development

1. Clone the repository, open the project in Android Studio, and run flutter pub get from the project root.

2. Complete the Firebase setup in Section 7.1. The package name, Firebase Android registration, OAuth client, and signing fingerprints must refer to the same application configuration.

3. Connect a device with USB debugging enabled, or start an emulator with Google Play services.

4. Run flutter run or use Run in Android Studio. Verify Google sign-in and onboarding before testing shared group records.

## 9.2 Build the Standalone APK

Confirm that the main AndroidManifest.xml declares INTERNET permission and uses the DefendIt application label. These are release checks, not a claim that the current manifest already contains them.

Run flutter clean, then flutter pub get, then flutter build apk --release. Copy build/app/outputs/flutter-apk/app-release.apk to the phone and install it. The installed application runs without a USB connection, laptop, or Android Studio; online features still require internet access.

The source configuration signs release builds with the debug key. Before distribution, use the intended release signing configuration and register its fingerprint with Firebase. Re-test authentication in the generated APK rather than relying only on a debug run.

## 9.3 Troubleshooting

| Problem | Check and corrective action |
|---|---|
| Google sign-in returns to login | Verify the SHA-1, Google provider, client ID, and package registration. Download matching google-services.json and rebuild. |
| Debug works but APK sign-in fails | Check INTERNET in the main manifest and the fingerprint of the key that actually signed the APK. |
| permission-denied | Verify deployed Firestore rules, authenticated identity, saved role, and group membership or adviser assignment. |
| Adviser code rejected | Confirm that the issued document exists in adviserCodes and is not claimed by another account. |
| Student number rejected | Check its format and whether another account already registered the number. |

# 10 Limitations Future Improvements and Testing

## 10.1 Limitations

Conflicts use device clocks. An incorrect clock can make stale work appear newer, and protecting pending local edits does not resolve every disagreement between devices. A queued edit is not evidence that another device has received the same state.

The four-group adviser limit and one-group-per-student rule depend on app checks and transactions and are not fully enforced by Security Rules. Registration confirms student-number uniqueness, but the number is self-declared and is not checked against an official class list.

Groups, slots, and booking require internet access. A consultation must be opened once online before its notes and action items can be used offline. Loaded records and pending operations remain device-specific until synchronization succeeds.

The source targets Android delivery; this guide does not establish testing on other platforms. Push notifications and group editing are not included. The separate test record, rather than this guide alone, should establish which build and devices passed verification.

## 10.2 Possible Improvements

- Cloud Functions for reminders and stronger enforcement of group limits.
- A pre-loaded class list to verify student numbers and an editable roster.
- Chapter document uploads and calendar export.

These are possible improvements, not features delivered by the current documentation.

## 10.3 Testing

The project requires a separate Test Cases Document with itemized UI, functional, and Firebase database and authentication integration tests. It should record inputs, expected outcomes, actual results, and the tested build. The following coverage follows the existing requirements.

| Area | Essential verification |
|---|---|
| Identity | Google sign-in, role registration, adviser-code claims, unique student numbers, persistent sessions, and sign-out. |
| Group coordination | Create and join a group, preview requests, accept within capacity, and reject conflicting acceptance. |
| Titles and milestones | Proposal changes and adviser feedback; task completion, submission conditions, approval, and revision locks. |
| Consultations | Publish and book slots, cancel eligible bookings, save notes, and maintain action items. |
| Offline synchronization | Keep local edits, retain failed operations, retry when connected, and confirm changes on another account or device. |
| Access and release | Reject unauthorized writes; install and verify the generated Android APK with its actual Firebase and signing configuration. |

Record failed expectations as defects. A successful screen action alone does not confirm backend authorization or synchronization. Verify stored records and the receiving account where the test concerns shared data.